

First Design Practice

1 Beach Balls

1.1 Description

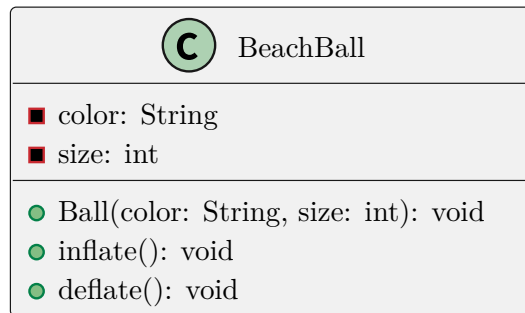
Create a class that represents some beach balls. Each ball has a color and a size. The balls can be inflated and deflated.



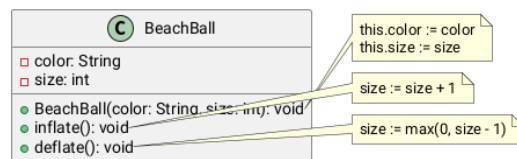
1.2 Type

Beach ball	<pre> BeachBall(color: String, size: int) inflate(): void deflate(): void </pre>
<pre> color: String size: int </pre>	<pre> BeachBall(color: String, size: int) this.color := color this.size := size end inflate(): void size := size + 1 end deflate(): void size := max(0, size - 1) end </pre>

1.3 Class diagram



1.4 Class diagram with method bodies



2 Person

2.1 Description

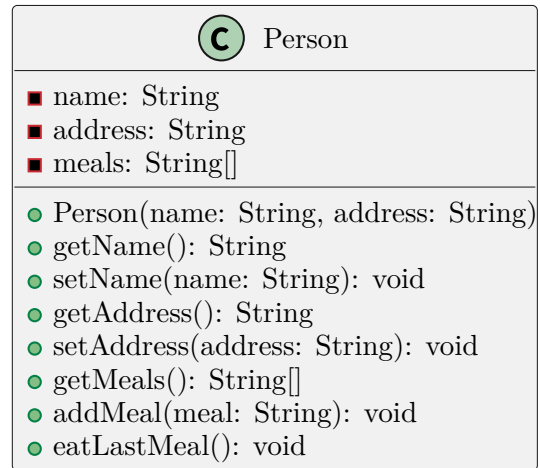
Create a class that represents a person. Each person has a name, an address, and a list of meals they have on them. They can eat the last meal in the list, or add a new meal. The class should have a constructor, getters, and setters.



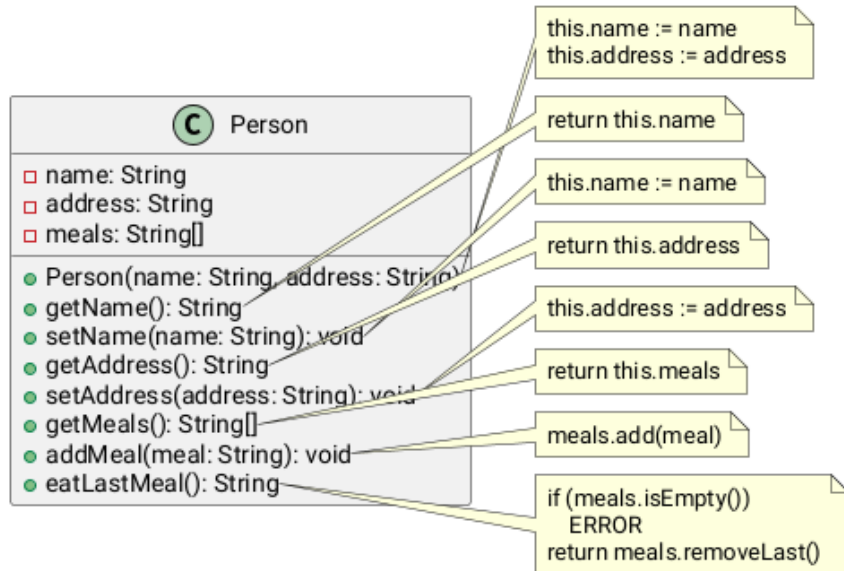
2.2 Type

Person	<pre>Person(name: String, address: String) getName(): String setName(name: String): void getAddress(): String setAddress(address: String): void getMeals(): String[] addMeal(meal: String): void eatLastMeal(): void</pre>
<pre>name: String address: String meals: String[]</pre>	<pre>Person(name: String, address: String) this.name := name this.address := address end getName(): String return this.name end setName(name: String): void this.name := name end getAddress(): String return this.address end setAddress(address: String): void this.address := address end getMeals(): String[] return this.meals end addMeal(meal: String): void meals.add(meal) end eatLastMeal(): String if (meals.isEmpty()) ERROR return meals.removeLast() end</pre>

2.3 Class diagram



2.4 Class diagram with method bodies



3 Exercise: Student

3.1 Description

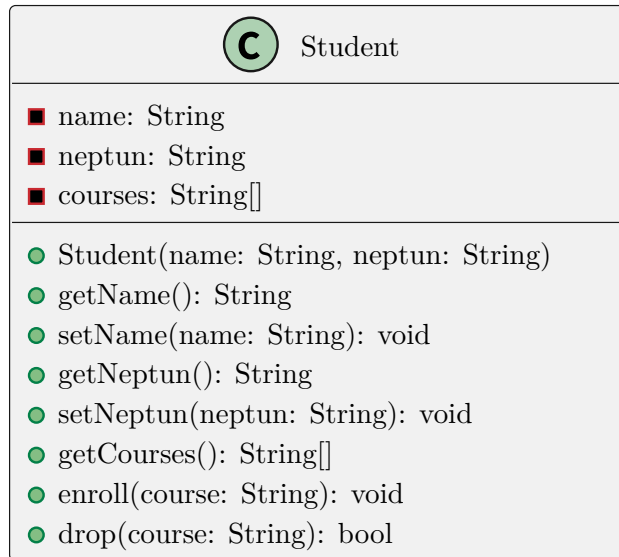
Create a class that represents a student. Each student has a name, a Neptun code, and a list of courses they attend. The student should be able to enroll into a new course and drop a course. The class should have a constructor, getters, and setters.



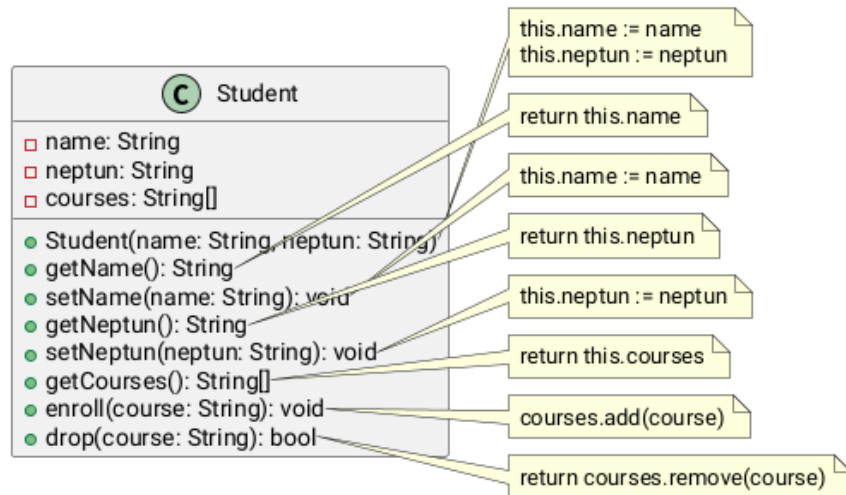
3.2 Type

Student	<pre>getName(): String setName(name: String): void getNeptun(): String setNeptun(neptun: String): void getCourses(): String[] enroll(course: String): void drop(course: String): bool</pre>
<pre>name: String neptun: String courses: String[]</pre>	<pre>Student(name: String, neptun: String) this.name := name this.neptun := neptun end getName(): String return this.name end setName(name: String): void this.name := name end getNeptun(): String return this.neptun end setNeptun(neptun: String): void this.neptun := neptun end getCourses(): String[] return this.courses end enroll(course: String): void courses.add(course) end drop(String course): bool return courses.remove(course) end</pre>

3.3 Class diagram



3.4 Class diagram with method bodies



4 Exercise: Wizard

4.1 Description

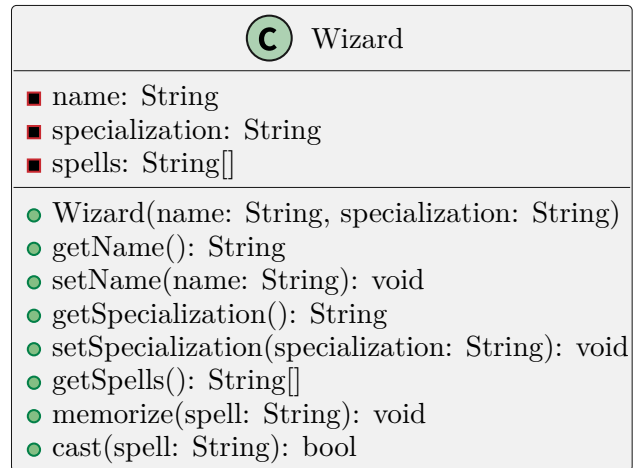
Create a class that represents a wizard. Each wizard has a name, a specialization (is a string, can be Illusion, Enchantment, Elemental, or Dark), and a list of spells she memorized. The wizard can commit a new spell into memory, and can also cast a spell, in which case she forgets it. The class should have a constructor, getters, and setters.



4.2 Type

Wizard	<pre>getName(): String setName(name: String): void getSpecialization(): String setSpecialization(specialization: String): void getSpells(): String[] memorize(spell: String): void cast(spell: String): bool</pre>
<pre>name: String specialization: String spells: String[]</pre>	<pre>Wizard(name: String, specialization: String) this.name := name this.specialization := specialization end getName(): String return this.name end setName(name: String): void this.name := name end getSpecialization(): String return this.specialization end setSpecialization(specialization: String): void this.specialization := specialization end getSpells(): String[] return this.spells end memorize(spell: String): void spells.add(spell) end cast(String spell): bool return spells.remove(spell) end</pre>

4.3 Class diagram



4.4 Class diagram with method bodies

